

Kennesaw State University

CSE 3502 Operating Systems

Project 2 - Pthread

Instructor: Kun Suo
Points Possible: 100
Difficulty: ★★★★★

S1 = This is a berry 🍷. This is cherry 🍒.
S2 = is

Part A: Transfer a sequential code into parallel (50 points)

Given two character strings s1 and s2. Write a Pthread program to find out the number of substrings, in string s1, that is exactly the same as s2.

For example, suppose `number_substring(s1, s2)` implements the function, then
`number_substring("abcdab", "ab") = 2`,
`number_substring("aaa", "a") = 3`,
`number_substring("abac", "bc") = 0`.

The size of s1 and s2 (n1 and n2) as well as their data are input by users. Assume that
`n1 mod NUM_THREADS = 0`
and
`n2 < n1/NUM_THREADS`.

The following is a sequential solution of the problem. `read_f()` reads the two strings from a file named "string.txt and `num_substring()` calculates the number of substrings.

<https://github.com/kevinsuo/CS3502/blob/master/project-pthread.c>

```
-----  
#include <stdlib.h>  
#include <stdio.h>  
#include <string.h>
```

```

#define MAX 10240

int total = 0;
int n1,n2;
char *s1,*s2;
FILE *fp;

int readf(FILE *fp)
{
    if((fp=fopen("strings.txt", "r"))==NULL){
        printf("ERROR: can't open string.txt!\n");
        return 0;
    }
    s1=(char *)malloc(sizeof(char)*MAX);
    if(s1==NULL){
        printf("ERROR: Out of memory!\n");
        return -1;
    }
    s2=(char *)malloc(sizeof(char)*MAX);
    if(s2==NULL){
        printf("ERROR: Out of memory!\n");
        return -1;
    }
    /*read s1 s2 from the file*/
    s1=fgets(s1, MAX, fp);
    s2=fgets(s2, MAX, fp);
    n1=strlen(s1); /*length of s1*/
    n2=strlen(s2)-1; /*length of s2*/
    if(s1==NULL || s2==NULL || n1<n2) /*when error exit*/
        return -1;
}

int num_substring(void)
{
    int i,j,k;
    int count;

    for (i = 0; i <= (n1-n2); i++){
        count=0;
        for(j = i,k = 0; k < n2; j++,k++){ /*search for the next string of size of n2*/
            if (*(s1+j)!=*(s2+k)){
                break;
            }
            else
                count++;
            if(count==n2)
                total++; /*find a substring in this step*/
        }
        return total;
    }
}

int main(int argc, char *argv[])
{
    int count;

    readf(fp);
    count = num_substring();
    printf("The number of substrings is: %d\n", count);
    return 1;
}

```

You can find an example of the “string.txt” here:

string.txt: <https://github.com/kevinsuo/CS3502/blob/master/strings.txt>

To compile the program with Pthread, use:

```
$ gcc project-pthread.c -o project-pthread.o -pthread
```

One possible output:

```
ksuo@ltsup66583mac ~/Desktop> ./project-pthread.o  
The number of substrings is: 55
```

(Different string text files, output is also different. For the above strings.txt, the output is 55)

Download the string.txt:

```
$ wget https://raw.githubusercontent.com/kevinsuo/CS3502/master/strings.txt
```

Write a parallel program using Pthread based on this sequential solution. Please set the thread number as **10** in your code. You can start with this template code:

```
https://github.com/kevinsuo/CS3502/blob/master/parallel-template.c
```

To compile the program with Pthread, use:

```
$ gcc file.c -o file.o -pthread
```

Here file refers to your source code name.

Expected output (the thread order can be random):

```
The number of substrings thread 1 find is ...  
The number of substrings thread 2 find is ...  
...  
The number of substrings thread 10 find is ...  
Total substrings is ...
```

HINT: Strings s1 and s2 are stored in a file named "string.txt". String s1 is evenly partitioned for `NUM_THREADS` threads to concurrently search for matching with string s2. After a thread finishes its work and obtains the number of local matchings, this local number is added into a global variable showing the total number of matched substrings in string s1. Finally, this total number is printed out. Please make sure the number of substrings of parallel program is the same as the serial program.

Part B: Compare performance between sequential and parallel (50 points)

Dataset

Use the provided file `strings_100x.txt`. It preserves the original two-line format in Part A: the first line is the original source string repeated consecutively 100 times, and the second line remains the search pattern is. The enlarged first string contains 479,800 characters. The correct substring total for this supplied file is 5,500.

https://github.com/kevinsuo/CS3502/blob/master/strings_100x.txt

Important: The original program's MAX value of 10,240 is too small for this dataset. Update the input code so the complete first line is read safely. Dynamic allocation (for example, `getline`) is recommended.

Required Implementations

Modify the above code in Part A and implement the following:

- Keep a sequential substring-counting function as the correctness baseline.
- Write one parallel implementation that accepts the number of worker threads as a command-line argument. Run it with exactly 2, 4, 8, 16, and 32 threads; do not maintain five separate source-code copies.
- Partition all valid starting positions from 0 through $n1 - n2$ so each position is examined exactly once. Overlapping matches and matches near partition boundaries must be counted correctly.
- Use `pthread_create()` and `pthread_join()`. Avoid data races by using per-thread local counts and combining them after all workers have joined, or by protecting shared updates correctly.
- Print the selected thread count, sequential total substring, parallel total substring, verification result, sequential time, parallel time, and calculated speedup for every measured configuration.

Correctness Verification

Your program must provide a `verify()` function that compares the sequential and parallel totals after every parallel run. A parallel timing result is valid only when the two totals are equal.

```
int verify(long sequential_total, long parallel_total)
{
    if (sequential_total == parallel_total) {
        printf("VERIFICATION: PASS\n");
        return 1;
    }
    printf("VERIFICATION: FAIL\n");
}
```

```

    return 0;
}

```

If verify() reports FAIL, print both totals, stop that experiment, and return a nonzero exit status. Do not report speedup for an incorrect result.

Timing Method

- Use clock_gettime (CLOCK_MONOTONIC, ...) for both sequential and parallel measurements.
- Time only the substring-search computation. Exclude file reading, allocation, result printing, and verify() from the measured interval.
- Run the sequential version at least five times and use its average execution time as T_{seq} .
- Run each parallel configuration at least five times and use its average execution time as T_p . Perform all runs on the same machine under similar conditions.
- Record elapsed time in nanoseconds or microseconds and use the same unit throughout the report.

Speedup

For p worker threads, calculate:

$$\text{Speedup } S(p) = \frac{T_{seq}}{T_p}$$

T_{seq} is the average sequential execution time, and T_p is the median parallel execution time using p threads. $S(p) > 1$ indicates a speedup; $S(p) < 1$ indicates that the parallel version was slower.

Required Results Table

Include a completed table like the following in the report:

Version	Threads	Avg time	Total	Verify	Speedup
Sequential	1		5,500	Baseline	1.00
Parallel	2			PASS / FAIL	
Parallel	4			PASS / FAIL	
Parallel	8			PASS / FAIL	

Version	Threads	Avg time	Total	Verify	Speedup
Parallel	16			PASS / FAIL	
Parallel	32			PASS / FAIL	

Analysis Questions

In your report, discuss and answer the following questions based on experiment results:

- Which thread count produced the lowest execution time?
- Did doubling the number of threads double the speedup? Explain why or why not.
- At what thread count did speedup stop improving, and how does this relate to the available CPU cores?
- What portions of the program remain sequential, and what overhead is introduced by pthreads?

() Submission format:

Please submit the following two files:

1. *One report file containing:*
 - a. *All screenshots and experiment results from Parts A and B.*
 - b. *A detailed discussion of your code logic and implementation*
2. *One ZIP file containing:*
 - a. *All code files from Parts A and B. The Code files should be organized clearly and logically*

Important: Submissions that do not follow the format above will not be graded.